

Learning Foresight without Explicit Trajectories for 3D Diffusion Policies

Zhongbo Zhang* Zaibin Zhang* Yifan Wang Changbo Yan Lijun Wang† Huchuan Lu
Dalian University of Technology

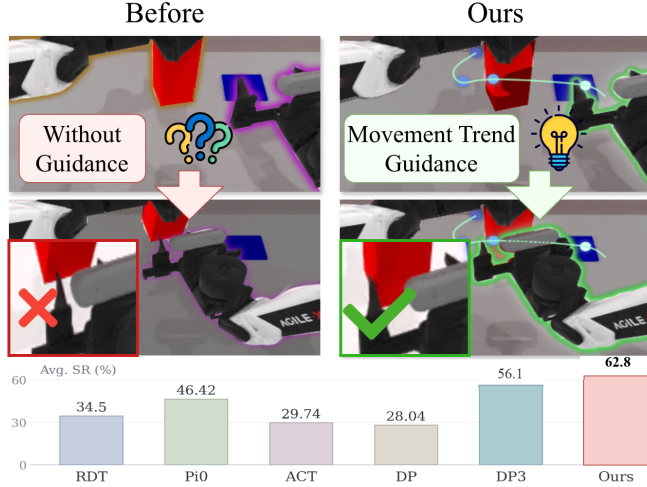


Fig. 1. Movement Trend Guidance for point-cloud diffusion policies. The latent trend provides soft geometric guidance that helps the policy organize future interaction while preserving observation-driven local action generation.

Abstract—3D diffusion policies are strong at generating geometrically grounded actions from current observations, but successful manipulation requires not only knowing what motion is feasible now, but also anticipating where the interaction is heading. Existing policies largely leave such foresight to emerge implicitly from action learning. We introduce Movement Trend Guidance, a simple but effective way to provide this foresight without introducing an explicit plan. From a short observation history, the policy learns a compact latent representation of interaction evolution. During training, sparse future gripper states supervise this representation; at inference, only the latent is retained as future-oriented conditioning alongside the current observation. The latent provides global conditioning for action generation, while an additional gated FiLM branch is used only at the UNet bottleneck. Despite adding only 3.52% more parameters to DP3, our method preserves the original dense-action and receding-horizon formulation and consistently improves upon DP3 across RoboTwin2.0, LIBERO-40, and DexArt. It reaches 62.8% vs. 56.1% in 50-task RoboTwin2.0 mixed training, 71.93% vs. 37.08% on LIBERO-40, and 72.0% vs. 49.0% on five real-robot tasks. These results show that a diffusion policy can benefit substantially from knowing where an interaction is heading, without being told exactly where to move. Code is available at <https://github.com/zhangzhongbo2213/movement-trend-guidance>.

I. INTRODUCTION

3D diffusion policies [1], [2], [3] provide a compelling formulation for robot manipulation: they map geometric observations directly to dense action sequences. This makes them well suited to precise control [1]. Yet manipulation is not determined solely by the motion that is feasible at the current instant. Before grasping, inserting, transferring an object, or operating an articulated mechanism, the robot must also understand *how the interaction is unfolding and what it is progressing toward*.

This creates a distinction between two kinds of information. Current geometry answers what action is appropriate now. Nonetheless, successful multi-step behavior additionally requires a notion of where the interaction is heading: which contact should occur next, or toward which configuration multiple effectors should coordinate. Standard diffusion policies do not explicitly separate these roles. Both local control and longer-range interaction structure must be recovered through the same action-learning objective.

A straightforward way to expose future structure is to predict waypoints, keyframes [4], [5], or trajectories [6]. However, doing so asks the predictor for more than the controller may actually need. A trajectory specifies where the robot should move, whereas action generation may only require a coarse indication of how the interaction is developing. Once a predicted future is converted into an intermediate control target, prediction errors can also constrain subsequent behavior, even when new observations suggest that the motion should be corrected.

We therefore consider a simpler use of future prediction: use it to teach the policy what future-relevant information to represent, rather than what future motion to execute. Our method, Movement Trend Guidance, predicts a compact latent representation from a short history of point clouds and robot states. During training, an auxiliary objective requires this latent to recover sparse future gripper states. During inference, only the latent representation, rather than the decoded future states, is retained as future-oriented conditioning for the action policy.

The architecture follows the same principle of minimal intervention. The trend latent enters the standard global-conditioning pathway, while only its additional gated FiLM modulation is restricted to the compressed bottleneck of the diffusion UNet. This small change adds no separate planning module and preserves dense action prediction and receding-horizon execution.

*Equal contribution. †Corresponding author.

Despite its simplicity, this design produces consistent gains across different manipulation settings. On 50-task RoboTwin2.0 mixed training, it improves the matched DP3 baseline from 56.1% to 62.8%; on LIBERO-40, from 37.08% to 71.93%; and on five real-robot tasks, from 49.0% to 72.0%. Controlled studies further show that future supervision improves the learned latent beyond additional representation capacity alone, while directly conditioning on explicit future points is less effective than using future prediction to shape the latent representation.

Our contributions are summarized as follows:

- We introduce a simple formulation of foresight for diffusion control: the policy learns not only what can be done at the current step, but also where the interaction is heading, without requiring an explicit planning target.
- We present a lightweight realization in which future states supervise a compact latent representation. The latent enters the standard global-conditioning pathway, while only its additional gated FiLM modulation is restricted to the UNet bottleneck.
- We provide broad empirical validation across RoboTwin 2.0, LIBERO-40, DexArt, and real-world robotic manipulation. The proposed method consistently improves 3D diffusion control while preserving the underlying diffusion-policy architecture.

II. RELATED WORK

A. Robot Manipulation

Sequence and action-chunking policies such as ACT [7], BeT [8], and VQ-BeT [9] improve imitation learning by modeling extended-horizon actions, while vision-language-action models extend this paradigm to multi-task policy learning [10], [11], [12], [13]. Diffusion Policy [14] shows conditional diffusion captures multimodal action distributions and generates coherent action chunks, while later work extends diffusion control to broader settings and faster inference [6], [15]. These methods provide strong dense action generators, but future interaction structure typically emerges implicitly from the action-learning objective. Our work preserves dense diffusion-based action generation while introducing a future-supervised latent representation of movement trend.

B. 3D Imitation Learning Policies

3D observations aid manipulation by exposing geometry, spatial relations, and viewpoint-robust structure [16], [17], [18], [19]. Prior 3D imitation learning methods use voxelized observations, neural fields, multiview features, or adaptive-resolution representations to predict actions or keyframes [20], [21], [22], [2], [23]. Many methods make intermediate structure explicit through keyframe pose prediction or prediction-and-planning, but rely on pose-level targets, explicit tracking, or planning interfaces distinct from direct dense action generation. DP3 [1] uses point-cloud observations with direct action diffusion, achieving strong sample efficiency and real-robot transfer. We follow DP3’s point-cloud diffusion

formulation, using sparse future-state supervision to shape a compact movement-trend latent jointly with action generation.

C. Future-Guided Policy Learning

Several policy-learning frameworks use future goals, sub-goals, keyframes, or trajectories to provide longer-horizon structure for manipulation. Recent diffusion-based approaches also introduce explicit guidance; HDP [24] leverages contact structure, while DTP [25] generates task-relevant 2D trajectories to guide manipulation. FLARE [26] and ForeDiffusion [27] introduce foresight through predictive representations of future visual observations. FLARE aligns policy features with future-observation latents, while ForeDiffusion predicts and injects a future-view representation. In contrast, our method operates directly in 3D space. From point-cloud and robot-state history, sparse task-space gripper states at multiple future offsets serve only as training-time supervision for a compact movement-trend latent. The decoded future states are discarded at inference, retaining only the latent as future-oriented conditioning alongside the current observation, with direct modulation restricted to the UNet bottleneck. Thus, our method differs in the future information modeled and its coupling to control, providing task-space foresight without future-view prediction or explicit future trajectories.

D. Conditional Modulation

Classifier-free guidance [28] controls conditioning strength in generative models, while ControlNet [29] introduces pathways for external conditions. Feature-wise modulation provides a lightweight mechanism for conditioning intermediate representations. These approaches motivate careful control of how auxiliary information interacts with a generative backbone. Here, the movement-trend latent provides a compact future-oriented condition rather than a dense control target. The trend latent enters the standard global-conditioning pathway, while its gated FiLM modulation is restricted to the UNet bottleneck. This allows future-supervised information to influence broader action-sequence structure while preserving the original pathway for observation-driven local refinement.

III. METHOD

This section presents movement-trend guidance for 3D diffusion policies. A standard observation-to-action policy must infer scene geometry, interaction intent, and precise local controls jointly. We instead expose a hidden latent from observation history that compresses future interaction information and provides soft geometric guidance while preserving flexible dense action generation without a decoded trajectory.

A. Problem Formulation

At time t , the policy receives an observation window of length N ,

$$\mathcal{O}_t = o_t - N + 1, \dots, o_t, \quad (1)$$

where each observation $o_i = (P_i, q_i)$ consists of a point cloud P_i and robot state q_i . As in DP3, the length- H prediction is

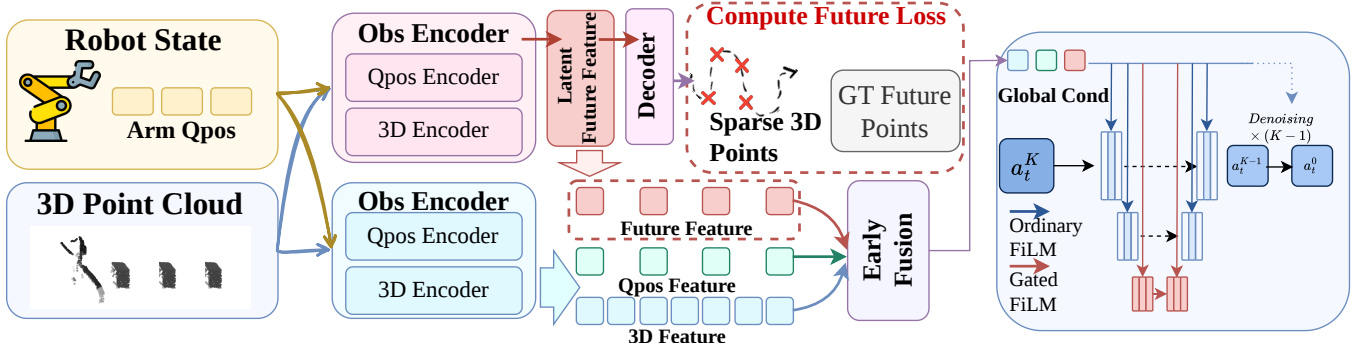


Fig. 2. Overview of Movement Trend Guidance. The sparse-point symbols denote the Cartesian components of future gripper targets; each target also contains the corresponding gripper state defined in Eq. (16). During training, the auxiliary decoder shapes the latent representation, while its outputs are discarded at inference and never supplied to the action policy. The latent enters the standard global-conditioning pathway, with an additional gated FiLM branch applied only at the UNet bottleneck.

aligned with the observation window and extends into the future:

$$\mathbf{A}t = at - N + 1, \dots, a_t, \dots, a_{t-N+H}. \quad (2)$$

The current action a_t is therefore at zero-based index $N - 1$. Following DP3’s receding-horizon setting, we set $N = 3$, $H = 8$ and execute $n_a = 6$ actions from this index before re-observing.

We represent future movement information with a compact latent rather than an explicit spatial trajectory. Let F_ϕ denote a lightweight future-information encoder mapping the observation window to

$$z_t = F_\phi(\mathcal{O}_t) \in \mathbb{R}^{d_z}, \quad d_z = 256. \quad (3)$$

The vector z_t is the latent preceding the auxiliary output head, which provides future gripper-target supervision during training but whose decoded values are never used as policy inputs. Thus, z_t represents a compressed movement trend rather than future coordinates. The action policy predicts action sequences from the observation and future trend latent:

$$\hat{\mathbf{A}}t = \Pi\theta(\mathcal{O}_t, z_t). \quad (4)$$

This factorization exposes future-dependent information without making it an intermediate control target. The latent summarizes interaction evolution, while the point-cloud encoder and action denoiser remain responsible for geometry and local control.

B. Latent Movement-Trend Representation

Each observation frame is encoded by a DP3-style point-cloud and state encoder,

$$e_i = E_{\text{trend}}(P_i, q_i), \quad i \in t - N + 1, \dots, t. \quad (5)$$

Before encoding, point-cloud coordinates and robot states are scaled to $[-1, 1]$ using per-dimension limits estimated from training data. This input normalization is distinct from future-target normalization. The encoded features and robot

states are concatenated temporally and passed through a two-hidden-layer MLP:

$$z_t = F_\phi([e_{t-N+1}; \dots; e_t], [q_{t-N+1}; \dots; q_t]). \quad (6)$$

The MLP hidden width is 256, and the latent is taken from the hidden layer immediately before the auxiliary future gripper-target head. The policy therefore receives the future-relevant representation, while the auxiliary head provides a compact training signal rather than a point-by-point tracking trajectory. The trend latent is first projected by a small MLP,

$$u_t = \psi(z_t). \quad (7)$$

For each observation frame, point-cloud and robot-state features are encoded independently and fused with the shared trend feature:

$$f_i^P = E_P(P_i), \quad f_i^q = E_q(q_i), \quad h_i = F_{\text{fuse}}([f_i^P; f_i^q; u_t]). \quad (8)$$

The same u_t is shared across all N observation frames, and the resulting features form the global diffusion condition,

$$c_t = [h_{t-N+1}; \dots; h_t]. \quad (9)$$

In the 50-task RoboTwin2.0 experiments, both *DP3* and *Ours* use the same learned 64-dimensional task embedding, projected into policy features to distinguish task identities. *DP3* otherwise follows vanilla DP3, whereas *Ours* additionally uses the movement-trend pathway and bottleneck modulation. Thus, task conditioning is matched, and controlled ablations retain the same embedding.

The latent trend acts as soft geometric guidance rather than a waypoint. The policy combines it with current point-cloud and robot-state observations to infer and execute local motions.

C. Action Diffusion with Bottleneck-Gated Guidance

Given a ground-truth action block $\mathbf{A}t$, the forward diffusion process [30] produces

$$x\tau = \sqrt{\alpha\tau}, \mathbf{A}t + \sqrt{1 - \alpha\tau}, \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (10)$$

The denoising network predicts the clean action sequence,

$$\hat{\mathbf{A}}t = D\theta(x\tau, \tau, c_t). \quad (11)$$

We use the sample-prediction objective,

$$\mathcal{L}_{\text{diff}} = \mathbb{E} \mathbf{A}t, \epsilon, \tau \left[\|D\theta(x\tau, \tau, c_t) - \mathbf{A}_t\|_2^2 \right]. \quad (12)$$

The latent-conditioned representation is available through the standard global-condition pathway. To regulate its direct influence on the denoising backbone, we additionally apply bottleneck-gated FiLM. Let x be the bottleneck feature and c the concatenated diffusion-time and observation condition:

$$\alpha = W_\alpha c, \quad \beta = W_\beta c, \quad \gamma = \sigma(W_\gamma c + b_\gamma), \quad (13)$$

$$\tilde{x} = (1 + \alpha) \odot x + \beta, \quad y = x + \gamma \odot (\tilde{x} - x). \quad (14)$$

The scale and bias projections are initialized to zero, making the additional branch an exact identity mapping initially, independent of the gate value. A negative gate bias further limits modulation as the projections begin to learn. The gated branch is restricted to the UNet bottleneck, while down-sampling and up-sampling blocks retain the ordinary latent-conditioned pathway. Thus, the latent remains available through global conditioning, with only direct additional modulation confined to the bottleneck.

D. End-to-End Training and Inference

We jointly optimize the trend encoder, latent predictor, latent embedding, observation encoder, and action diffusion model with a single end-to-end objective. The auxiliary decoder maps the latent to $K = 4$ future gripper targets,

$$\hat{\mathbf{P}}t = G\omega(z_t) \in \mathbb{R}^{K \times 2 \times 4}, \quad \Delta = 5, 10, 15, 20. \quad (15)$$

For offset δ_k and gripper slot g , the target $\mathbf{p}_{t+\delta_k}^{(g)} \in \mathbb{R}^4$ contains Cartesian end-effector position (x, y, z) and one scalar gripper state. The two slots correspond to the left and right grippers in bimanual tasks; for single-arm LIBERO tasks, the target is duplicated across both slots to preserve the tensor interface.

Offsets index the trajectory supplied by each benchmark. In RoboTwin2.0, stride-4 sampling changes only training-window anchors, so offsets remain measured in original demonstration frames. In LIBERO-40, they index the stride-4 materialized trajectory. Targets use each benchmark’s stored Cartesian frame and enter the loss directly without separate future-target normalization. Offsets beyond a demonstration are clipped to its final frame. We use the elementwise mean-squared error

$$\mathcal{L}_{\text{future}} = \frac{1}{8K} \sum_k k = 1^K \sum_{g=1}^2 \left| \hat{\mathbf{p}}^{(g)}t + \delta_k - \mathbf{p}^{(g)}t + \delta_k \right|^2, \quad (16)$$

and optimize

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_{\text{future}} \mathcal{L}_{\text{future}}, \quad \lambda_{\text{future}} = 1.0. \quad (17)$$

The decoder produces $4 \times 2 \times 4 = 32$ scalars as a compact geometric training signal, not a waypoint sequence or tracking

target. The parameter-matched ablation sets $\lambda_{\text{future}} = 0$ while retaining the latent architecture.

During inference, the policy denoises Gaussian noise under c_t and executes

$$\mathbf{a}_t^{\text{exec}} = \hat{\mathbf{A}}t[N - 1 + n_a] = \hat{a}_t, \dots, \hat{a}_t + n_a - 1. \quad (18)$$

After each executed subsequence, the robot obtains a new calibrated point-cloud observation, recomputes z_t , and replans. The latent is refreshed every receding-horizon cycle and never directly converted into a command or tracking path.

IV. EXPERIMENTS

We evaluate the proposed movement-trend-guided DP3 on simulation benchmarks and real-robot manipulation tasks. We evaluate on RoboTwin2.0 [31], a large-scale bimanual manipulation benchmark with 50 diverse tasks. We further evaluate and analyze our method in the distinct environments of LIBERO-40 [32] and DexArt [33] to assess its generality. Finally, we evaluate the method on five SO101 real-robot manipulation tasks.

A. Experimental Setup

We evaluate on RoboTwin2.0, LIBERO-40, DexArt, and five SO101 real-robot tasks. For RoboTwin2.0, each task contains 50 demonstrations and is evaluated over 100 episodes. We first reproduce the official RoboTwin2.0 setting. We also train one policy jointly on all 50 tasks to assess multi-task capability. This mixed-training setting uses 50 demonstrations per task. We retain every fourth frame as a training-window anchor while preserving consecutive frames within each sampled window. The policy is trained for 3000 epochs, matching the original training schedule. Evaluation strictly follows the official protocol for a fair comparison.

For LIBERO-40, each task contains 50 demonstrations. We use trajectories materialized with stride-4 temporal subsampling and train for 1000 epochs. Checkpoints saved every 100 epochs are evaluated with rollout seeds 7, 17, and 27, using 50 episodes per task and seed. DexArt uses 100 demonstrations per task and 3000 training epochs. Policies are evaluated every 200 epochs with 20 episodes per task, and the reported score averages the best five evaluation checkpoints. This benchmark-specific aggregation differs from the fixed epoch-1000 LIBERO report. Each real-robot task uses 50 demonstrations and 20 evaluation rollouts. Our DP3 reproductions and movement-trend variants use AdamW with a learning rate of 1.0×10^{-4} . We report success rate as the primary metric.

Our method contains 271.67M parameters, adding 9.23M (+3.52%) over the 262.43M-parameter DP3 baseline. On an NVIDIA RTX 4090 with batch size 1 and 10 DDIM denoising steps, this increases mean inference latency only from 50.28 ms to 50.90 ms (+1.23%).

B. Main Results on RoboTwin2.0

Table I retains the original RoboTwin2.0 category-level comparison and appends the new 50-task mixed-training

TABLE I

ROBOTWIN2.0 CATEGORY-LEVEL SUCCESS RATES (%). THE ORIGINAL COMPARISON ROWS ARE RETAINED. THE STARRED ROWS ARE THE NEWLY SUPPLIED 50-TASK MIXED-TRAINING RESULTS FOR $DP3^*$ AND $Ours^*$, USING 50 DEMONSTRATIONS PER TASK AND 100 EVALUATION EPISODES PER TASK. BOTH STARRED POLICIES RECEIVE THE SAME LEARNED 64-DIMENSIONAL TASK EMBEDDING; $DP3^*$ OTHERWISE RETAINS VANILLA DP3 CONDITIONING, WHILE $Ours^*$ ADDITIONALLY INCLUDES THE PROPOSED TREND PATHWAY AND BOTTLENECK MODULATION. VALUES ARE AVERAGED OVER THE TASKS IN EACH CATEGORY; THE FINAL COLUMN IS THE UNWEIGHTED AVERAGE OVER ALL 50 TASKS.

Method	Single-Arm Handling	Articulated Interaction	Bimanual Handover	Pick & Transport	Precise Placement	Multi-Object Stacking	Tool Use & Precision	Overall Average
RDT	67.4	51.9	67.5	22.0	20.2	37.5	17.0	34.5
π_0	80.0	61.3	71.5	42.3	32.1	54.0	18.0	46.4
ACT	70.1	38.4	63.5	17.3	14.5	38.8	12.2	29.7
DP	59.3	32.0	31.5	22.8	19.6	32.8	10.8	28.0
DP3	84.9	71.3	85.0	54.4	45.5	41.3	25.2	55.2
Ours	88.9	78.0	95.0	61.9	52.0	51.0	29.0	61.7
$DP3^*$	79.9	72.4	85.5	55.6	49.3	37.8	27.2	56.1
$Ours^*$	87.7	80.0	94.0	62.9	54.8	48.3	29.8	62.8

results as the final two rows. Each entry is a success rate (%), averaged over the tasks in the corresponding category.

$Ours^*$ reaches 62.8%, a +6.7-point gain over $DP3^*$, and improves on 38 of the 50 tasks; the remaining tasks show broadly comparable performance overall. Because the two starred systems use the same task embedding, this comparison directly evaluates the addition of the trend pathway and bottleneck modulation. Component-level attribution is further examined in the controlled ablations in Section V, where the task embedding is held fixed. The largest task-level gains occur on contact-rich and multi-stage tasks such as rotate_QRcode (+37), move_playingcard_away (+21), move_pillbottle_pad (+22), and place_shoe (+26).

C. Evaluation Across Environments

We evaluate the method in two distinct simulated environments, LIBERO-40 and DexArt, using their respective official task definitions and evaluation protocols.

1) **LIBERO-40 Point-Cloud Evaluation:** To compare our method directly with the point-cloud baseline, we evaluate both policies on the 40-task LIBERO suite. The four suites (LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, and LIBERO-10) use the same 50 training demonstrations per task, the same calibrated point-cloud preprocessing, and the same evaluation initialization protocol. For each checkpoint saved every 100 epochs from epoch 100 to 1000, evaluation uses rollout seeds 7, 17, and 27 with 50 episodes per task and seed, totaling 6,000 episodes per checkpoint. Figure 3 shows the overall success rate across these saved checkpoints, while Table II reports the suite-level and overall epoch-1000 results as mean $\pm$ sample standard deviation across the three seed-level success rates.

Our method substantially outperforms DP3 on all four suites, reaching $71.93\% \pm 0.49\%$ overall at epoch 1000 and a +34.85-point gain over DP3. The corresponding LIBERO guidance ablations are analyzed in Section V-B. The largest

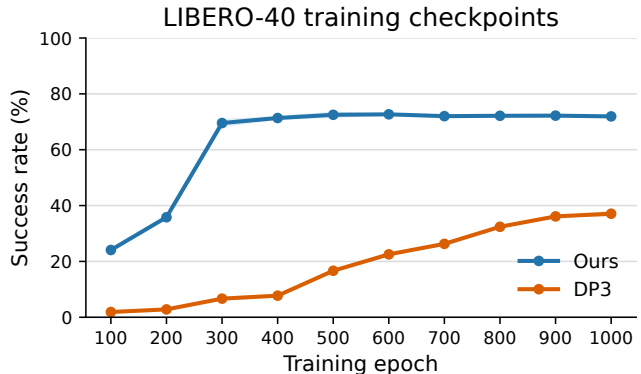


Fig. 3. LIBERO-40 point-cloud evaluation. Overall success rate for checkpoints saved every 100 epochs from epoch 100 to 1000.

absolute gain is on LIBERO-Goal. The narrow seed deviations indicate that the advantage is consistent across the three evaluation seeds rather than being driven by one rollout stream.

2) **DexArt:** We further evaluate the method on four DexArt tasks. DexArt features dexterous articulated object manipulation and differs from RoboTwin2.0 in robot morphology, task structure, and object categories.

Our method reaches 59.25% average success on DexArt, improving over DP3 by 7.25 points and achieving the highest average among the compared methods.

D. Real-Robot Experiments

We further conduct real-robot experiments on the SO101 dual-arm platform on five manipulation tasks: push cube, stack bowls, stack cubes, lift basket, and handover bottle. The policy receives point clouds reconstructed from Intel RealSense RGB-D observations: synchronized stereo depth is deprojected using the calibrated camera intrinsics and transformed into the robot coordinate frame with camera-

TABLE II

LIBERO-40 POINT-CLOUD EVALUATION AT EPOCH 1000. VALUES ARE SUCCESS RATES (%) REPORTED AS MEAN $\pm$ SAMPLE STANDARD DEVIATION ACROSS ROLLOUT SEEDS 7, 17, AND 27; FIG. 3 USES THE SAME AGGREGATION FOR CHECKPOINTS SAVED EVERY 100 EPOCHS. EACH SUITE CONTAINS 10 TASKS AND 1,500 EVALUATION EPISODES. OVERALL CONTAINS ALL 40 TASKS AND 6,000 EPISODES.

Suite	Tasks	DP3	Ours	Improvement	Episodes
LIBERO-Spatial	10	26.40 $\pm$ 1.31	58.20 $\pm$ 0.72	+31.80 $\pm$ 1.22	1,500
LIBERO-Object	10	67.53 $\pm$ 1.14	93.53 $\pm$ 0.70	+26.00 $\pm$ 1.44	1,500
LIBERO-Goal	10	8.73 $\pm$ 0.76	77.53 $\pm$ 2.32	+68.80 $\pm$ 2.62	1,500
LIBERO-10	10	45.67 $\pm$ 1.80	58.47 $\pm$ 0.64	+12.80 $\pm$ 2.42	1,500
Overall	40	37.08 $\pm$ 1.21	71.93 $\pm$ 0.49	+34.85 $\pm$ 1.30	6,000

TABLE III

DEXART SUCCESS RATES (%) WITH REPRESENTATIVE BASELINES FROM THE DEXART EVALUATION SETTING. OUR RESULT AVERAGES THE BEST FIVE CHECKPOINTS UNDER THE EVALUATION SCHEDULE DESCRIBED IN SECTION IV-A. BEST RESULTS ARE BOLDDED AND SHADED IN BLUE.

Method	DexArt				
	faucet	laptop	toilet	bucket	Avg.
IBC	7	3	0	14	6.0
BCRNN	1	3	0	5	2.3
H ³ DP	34	81	70	28	53.3
FreqPolicy	30	85	77	25	54.3
DP	23	69	58	20	42.5
SimpleDP3	26	79	63	22	47.5
DP3	36	77	70	25	52.0
VITA	39	82	72	28	55.3
Ours	41	87	76	33	59.25

TABLE IV

REAL-ROBOT SUCCESS RATES (%) ON FIVE TASKS. BEST RESULTS ARE BOLDDED AND SHADED.

Method	Push Cube	Stack Bowls	Stack Cubes	Lift Basket	Handover Bottle	Avg.
DP	40	15	60	20	30	33.0
SimpleDP3	40	25	70	30	50	43.0
DP3	50	30	75	35	55	49.0
Ours	75	75	85	45	80	72.0

to-robot extrinsics. Each task uses 50 demonstrations and is evaluated over 20 rollouts. Table IV reports the resulting success rates. Our method reaches 72.0% average success, compared with 49.0% for DP3 and 43.0% for SimpleDP3.

E. Failure Cases

Some tasks show only modest improvements over DP3. Our failure analysis identifies a common geometric cause: the robot arm can occlude most of the manipulated object, leaving an incomplete point cloud. The missing geometry affects both policies and can also perturb the movement-trend estimate used by our method. The effect is most visible in tasks that require precise contact, grasp alignment, or placement. In these cases, a trend estimate based on partial observations may provide only limited additional information. This behavior is consistent with our design: the movement-trend latent provides soft guidance, so the policy must still combine it with the current point-cloud and robot-state observations to infer local actions.

Our real-robot experiments demonstrate transfer in the evaluated SO101 setting; broader validation across diverse

objects, scenes, and perturbations remains an important direction for future work.

V. ABLATION STUDIES

We ablate key design choices on RoboTwin2.0. Unless otherwise noted, the DP3-based ablation variants are trained jointly on the complete 50-task suite using the same mixed-task sampling, demonstration budget, and evaluation protocol as the main experiment. Each reported value therefore evaluates one shared-policy run rather than an independently trained task-specific policy. The ACT experiment in Section V-E is a separate six-task validation. All variants use the default future-target offsets unless otherwise specified. Our implementation uses a learned latent trend representation rather than an explicit sparse-point sequence. The ablations therefore focus on the latent guidance path, its training signal, its future-target offsets, and the architecture used to inject it into the diffusion model. Figure 4 summarizes the ablation dimensions under this shared-policy protocol.

A. Guidance Construction and Training

This group validates the latent guidance path, bottleneck-gated modulation, and training objective while keeping joint training over all 50 RoboTwin2.0 tasks fixed. Removing gated FiLM leaves the latent trend available through early fusion and reduces the model to a DP3-style diffusion policy with an enriched global condition. The comparison tests whether the latent is useful merely as extra representation capacity or whether bottleneck regulation provides a distinct benefit. Additional parameter-matched variants replace the default MLP trend predictor with a diffusion predictor or remove input normalization while preserving the shared 50-task protocol. The latter bypasses limits normalization for the point-cloud and robot-state inputs of the trend predictor; action normalization and the unnormalized future targets are unchanged. These comparisons test whether the observed gain depends on the specific predictor and preprocessing choices.

B. LIBERO-40 Guidance Ablations

To test whether the same design choices transfer beyond RoboTwin2.0, we repeat the comparison on LIBERO-40 under a fixed point-cloud protocol. Each variant uses 50 demonstrations per task, stride-4 temporal subsampling, and 1000 training epochs. Evaluation at epoch 1000 uses rollout seeds 7, 17, and 27 with 50 episodes per task and seed.

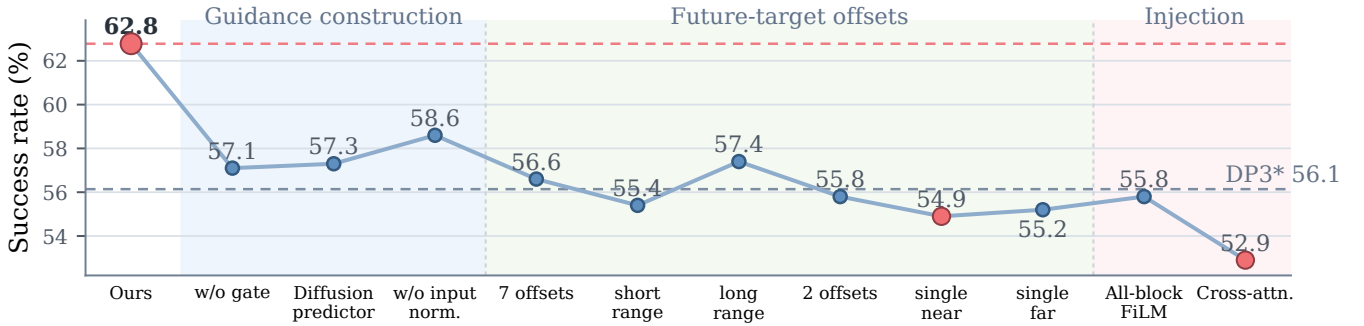


Fig. 4. Ablation dimensions under the 50-task mixed-training setting. We vary guidance construction, future-target offsets, and the scope of bottleneck-gated injection; every point is obtained from a shared policy trained jointly on all 50 RoboTwin2.0 tasks. The observation-history length remains fixed.

TABLE V

COMPACT LIBERO-40 ABLATION RESULTS AT EPOCH 1000. VALUES ARE FOUR-SUITE SUCCESS RATES REPORTED AS MEAN $\pm$ SAMPLE STANDARD DEVIATION ACROSS ROLLOUT SEEDS 7, 17, AND 27.

Method	Future guidance	SR (%)
DP3	None	37.08 $\pm$ 1.21
Ours	Latent trend + future loss	71.93 $\pm$ 0.49
Parameter-Matched Latent	Same latent, no future loss	44.87 $\pm$ 0.33
Explicit Future-Point Conditioning	Points at 5, 10, 15, 20 embedded in condition	65.28 $\pm$ 0.19

Table V reports the four-suite success rate as the mean $\pm$ sample standard deviation across the three seeds. The complete model achieves the highest result, $71.93 \pm 0.49\%$. The parameter-matched latent variant without future supervision reaches $44.87 \pm 0.33\%$, only $+7.79$ points above DP3. Adding future supervision yields a further $+27.06$ points. The complete model also exceeds explicit future-point conditioning by $+6.65$ points. These results show that future-supervised representation learning, rather than additional latent capacity alone, drives most of the improvement.

C. Future-Target Offset Configuration

This group varies the future offsets used by the auxiliary supervision while keeping the three-frame observation history fixed in the same 50-task mixed-training run. Besides the default $\{5, 10, 15, 20\}$, we evaluate seven offsets $\{5, 10, 15, 20, 25, 30, 35\}$, short-range offsets $\{2, 4, 6, 8\}$, long-range offsets $\{20, 30, 40, 50\}$, two offsets $\{5, 15\}$, a single near offset $\{5\}$, and a single far offset $\{30\}$. These values are original demonstration-frame offsets; stride-4 sampling changes only the training-window anchors. Short-range targets can be redundant with the observation window, whereas overly long ranges can dilute the compact representation with details that are better handled by the action diffusion model. The default offsets balance near-term contact relevance and longer-horizon movement direction. These targets shape the latent during training; they are not intermediate coordinates that the executed actions must reach.

D. Bottleneck-Gated FiLM Injection

This group examines how latent movement-trend guidance enters the denoising backbone, with every injection variant trained jointly on all 50 tasks. The full method applies gated FiLM only at the UNet bottleneck, while the standard conditional pathway carries the latent-conditioned global representation through the network. Applying gated FiLM at every UNet block lets the high-level latent directly perturb layers responsible for local action refinement, while cross-attention creates an even stronger coupling between denoising features and the trend condition. Bottleneck-only gated FiLM reaches 62.8%, compared with 55.8% for all-block gated FiLM and 52.9% for cross-attention under the same protocol. These experiments test the conservative injection hypothesis: latent movement-trend guidance should shape global action structure without dominating local denoising. All controlled variants use the same 64-dimensional task embedding so that one shared latent trend encoder can distinguish task-specific interaction semantics while being trained on the full 50-task mixture.

E. Transfer Across Policy Architectures

To test whether movement-trend conditioning is specific to the DP3 diffusion UNet, we conduct a separate validation with ACT, a transformer-based imitation policy, on the six RoboTwin2.0 tasks listed in Table VI. ACT and ACT + Trend are each trained jointly across these six tasks for 10,000 steps. This six-task training setting differs from the official benchmark ACT comparison retained in Table I, so the per-task ACT values in the two tables are not directly comparable. Both variants use the same mixed-training and evaluation protocol. The movement trend is predicted from the current observation and supplied as an additional condition; no ground-truth future state is used at test time.

In this six-task validation, movement-trend conditioning improves ACT on all evaluated tasks after 10,000 training steps. The mean success rate increases from 24.00% to 51.67%. This result suggests that the learned trend can provide useful guidance beyond the DP3 diffusion backbone. We limit this observation to the evaluated ACT setting and do not claim that the same gain holds for every policy architecture.

TABLE VI

ACT WITH AND WITHOUT MOVEMENT-TREND CONDITIONING IN A SIX-TASK ROBOTWIN2.0 VALIDATION. BOTH POLICIES ARE TRAINED JOINTLY ON THE SIX LISTED TASKS FOR 10,000 STEPS (SUCCESS RATE %).

Task	ACT	ACT + Trend	Gain
handover_block	42	49	+7
place_cans_plasticbox	16	48	+32
stack_bowls_two	25	73	+48
move_can_pot	22	67	+45
click_alarmsclock	32	63	+31
hanging_mug	7	10	+3
Mean	24.00	51.67	+27.67

VI. CONCLUSION

We presented Movement Trend Guidance for 3D diffusion policy learning, addressing the gap between dense action generation and implicit future interaction intent. Our method extracts a compact latent trend representation from a short history of point-cloud and robot-state observations and uses it as soft global conditioning. Because the latent is not decoded into a waypoint sequence or tracking target, the action policy can use future information while retaining the freedom to correct local motion from current geometry. The latent is available through the standard global-conditioning pathway, while an additional gated FiLM branch is restricted to the compressed UNet bottleneck. The trend encoder and action diffusion policy are optimized jointly with end-to-end action diffusion training and an auxiliary future gripper-target signal; a parameter-matched ablation removes this signal while retaining the same latent architecture. Experiments across RoboTwin2.0, LIBERO-40, DexArt, and five SO101 real-robot tasks demonstrate the value of this compact guidance mechanism. The real-robot pipeline obtains calibrated point clouds from Intel RealSense RGB-D observations. Overall, latent movement trends provide a lightweight way to add foresight to 3D diffusion policies without turning future information into a hard plan.

REFERENCES

- [1] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3d diffusion policy: Generalizable visuomotor policy learning via simple 3d representations,” *arXiv preprint arXiv:2403.03954*, 2024.
- [2] T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki, “Act3d: 3d feature field transformers for multi-task robotic manipulation,” in *Conference on Robot Learning*, 2023.
- [3] Z. Hong, S. Wu, H. Cui, B. Zhao, R. Ji, Y. He, H. Zhang, Z. Ke, J. Wang, G. Zhang, *et al.*, “R3d: Revisiting 3d policy learning,” *arXiv preprint arXiv:2604.15281*, 2026.
- [4] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, *et al.*, “Causal world modeling for robot control,” *arXiv preprint arXiv:2601.21998*, 2026.
- [5] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, *et al.*, “pi0.7: a steerable generalist robotic foundation model with emergent capabilities,” *arXiv preprint arXiv:2604.15483*, 2026.
- [6] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *International Conference on Machine Learning*, 2022.
- [7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023.
- [8] N. M. M. Shafiullah, Z. J. Cui, A. Altanzaya, and L. Pinto, “Behavior transformers: Cloning k modes with one stone,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [9] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, “Behavior generation with latent actions,” *arXiv preprint arXiv:2403.03181*, 2024.
- [10] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choro-manski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [11] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, “Openvla: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [12] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [13] A. O’Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandlekar, A. Jain, *et al.*, “Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 6892–6903.
- [14] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *arXiv preprint arXiv:2303.04137*, 2023.
- [15] G. Lu, Z. Gao, T. Chen, W. Dai, Z. Wang, and Y. Tang, “Manicm: Real-time 3d diffusion policy via consistency model for robotic manipulation,” *arXiv preprint arXiv:2406.01586*, 2024.
- [16] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 652–660.
- [17] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” *Advances in neural information processing systems*, vol. 30, 2017.
- [18] H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun, “Point transformer,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 16 259–16 268.
- [19] M. Shridhar, L. Manuelli, and D. Fox, “Perceiver-actor: A multi-task transformer for robotic manipulation,” *arXiv preprint arXiv:2209.05451*, 2022.
- [20] Y. Ze, G. Yan, Y.-H. Wu, A. Macaluso, Y. Ge, J. Ye, N. Hansen, L. E. Li, and X. Wang, “Gnfactor: Multi-task real robot learning with generalizable neural feature fields,” in *Conference on Robot Learning*, 2023.
- [21] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox, “Rvt: Robotic view transformer for 3d object manipulation,” in *Conference on Robot Learning*, 2023.
- [22] A. Goyal, V. Blukis, J. Xu, Y. Guo, Y.-W. Chao, and D. Fox, “Rvt-2: Learning precise manipulation from few demonstrations,” *arXiv preprint arXiv:2406.08545*, 2024.
- [23] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki, “3d diffuser actor: Policy diffusion with 3d scene representations,” *arXiv preprint arXiv:2402.10885*, 2024.
- [24] D. Wang, C. Liu, F. Chang, and Y. Xu, “Hierarchical diffusion policy: Manipulation trajectory generation via contact guidance,” *arXiv preprint arXiv:2411.12982*, 2024.
- [25] S. Fan, Q. Yang, Y. Liu, K. Wu, Z. Che, Q. Liu, and M. Wan, “Diffusion trajectory-guided policy for long-horizon robot manipulation,” *arXiv preprint arXiv:2502.10040*, 2025.
- [26] R. Zheng, J. Wang, S. Reed, J. Bjorck, Y. Fang, F. Hu, J. Jang, K. Kundalia, Z. Lin, L. Magne, A. Narayan, Y. L. Tan, G. Wang, Q. Wang, J. Xiang, Y. Xu, S. Ye, J. Kautz, F. Huang, Y. Zhu, and L. Fan, “FLARE: Robot learning with implicit world modeling,” in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 3952–3971.
- [27] W. Xie, Y. Ding, Y. He, L. Wang, B. Bai, Z. Zhao, C. Wang, and F. R. Yu, “ForeDiffusion: Foresight-conditioned diffusion policy via future view construction for robot manipulation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 22, 2026, pp. 18 665–18 673.
- [28] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.

- [29] L. Zhang, A. Rao, and M. Agrawala, “Adding conditional control to text-to-image diffusion models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.
- [30] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [31] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, *et al.*, “Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” *arXiv preprint arXiv:2506.18088*, 2025.
- [32] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [33] C. Bao, H. Xu, Y. Qin, and X. Wang, “Dexart: Benchmarking generalizable dexterous manipulation with articulated objects,” *arXiv preprint arXiv:2305.05706*, 2023.
- [34] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “Rdt-1b: a diffusion foundation model for bimanual manipulation,” *arXiv preprint arXiv:2410.07864*, 2024.

APPENDIX I

FULL PER-TASK RESULTS ON ROBOTWIN2.0

The numerical table in this appendix reports the supplied task-wise results for $DP3^*$ and $Ours^*$, each trained jointly on all 50 RoboTwin2.0 tasks.

Table VII reports the success rate (%) of all methods on each of the 50 RoboTwin2.0 tasks. The starred columns are the 50-task mixed-training results; the other columns retain the official benchmark comparisons. Tasks are grouped by category. Category averages and the overall average are included. The best result in each row is shown in **bold** with blue shading.

APPENDIX II

EXPERIMENTAL DETAILS AND ADDITIONAL ROBOTWIN2.0 ANALYSIS

RoboTwin2.0 [31] contains 50 bimanual manipulation tasks organized around single-arm object handling, articulated object interaction, bimanual handover, pick-and-transport, precise placement, multi-object stacking, and tool use with fine-grained precision. Each task uses 50 expert demonstrations for training and is evaluated over 100 episodes. We separately and strictly reproduce the official RoboTwin2.0 setting, and additionally evaluate a 50-task mixed-training setting to assess the model’s multi-task capability. On RoboTwin2.0, we compare against five representative methods spanning different policy families: RDT [34], π_0 [12], ACT [7], Diffusion Policy (DP) [14], and DP3 [1]. All baseline numbers are taken from the official RoboTwin2.0 benchmark evaluation. For our method, the movement-trend encoder and action diffusion policy are optimized jointly throughout end-to-end training. The full latent-guidance implementation uses an auxiliary future gripper-target loss; the parameter-matched LIBERO ablation removes this loss while keeping the latent architecture fixed. The latent predictor uses a 256-dimensional hidden representation. Both starred policies receive the same learned 64-dimensional task embedding, which is projected into the observation features before global conditioning of the action diffusion policy. $DP3^*$ otherwise follows vanilla DP3, while $Ours^*$ adds the trend pathway and bottleneck modulation; controlled ablations retain the same task embedding for component attribution. Both the

trend encoder and action diffusion policy use AdamW with a learning rate of 1.0×10^{-4} .

All simulation data used in our experiments is generated from the corresponding benchmark simulators rather than collected from physical robots. Specifically, RoboTwin2.0 demonstrations and evaluation rollouts are generated with the RoboTwin2.0 simulator, and the DexArt experiments are generated with the DexArt simulator. For real-robot experiments, we manually collect demonstrations on the SO101 dual-arm platform through teleoperation, as described below.

For DexArt, each task uses 100 demonstrations. Policies are trained for 3000 epochs, evaluated every 200 epochs over 20 episodes, and reported by averaging the best five evaluation checkpoints. This benchmark-specific aggregation differs from the fixed epoch-1000 LIBERO report.

For real-robot evaluation, we use the open-source SO101 dual-arm platform on push cube, stack bowls, stack cubes, lift basket, and handover bottle. The platform is observed with Intel RealSense RGB-D cameras. Each camera provides a synchronized RGB stream and stereo-depth stream; the depth image is deprojected with the calibrated camera intrinsics and transformed into the policy coordinate frame with the camera-to-robot extrinsics. After invalid-depth filtering and point sampling, the resulting calibrated point cloud is used as the same 3D observation modality as in simulation. The corresponding real-robot experimental setup is shown in Figure 5. We collect 50 real-robot demonstrations for each task using LeRobot teleoperation and evaluate each task over 20 rollouts.

The task-level trends in Table VII align with the intended role of movement trend guidance. Bimanual handover improves by +8.5 points, where the two arms must coordinate approach, grasp, transfer, and release. Latent trend encoding captures future interaction intent for the arms, providing soft coordination cues that are not explicit in the observation window alone. Multi-object stacking shows the largest category-level gain (+10.5), since stacking requires target height and ordering information several steps ahead. Single-arm object handling (+7.9), articulated object interaction (+7.6), and pick-and-transport (+7.3) similarly involve multi-phase behavior where compressed future information can guide smoother action generation. The gains are smaller on precise placement (+5.5) and tool-use tasks (+2.6), where success depends strongly on fine-grained local geometry.

We further partition the 50 RoboTwin2.0 tasks by $DP3^*$ baseline success rate. Among the 11 tasks with $DP3^*$ success rate below 30%, our method improves on 10 and achieves an average gain of +6.1 points. For the five tasks with $DP3^*$ success rates from 30% to below 40%, the average gain is +12.0 points. Among the 21 tasks with $DP3^*$ success rates from 40% to 80%, the average gain is +9.8 points, where movement trend guidance provides the most complementary value to the existing geometric perception. Among the 13 tasks with $DP3^*$ success rates above 80%, the average change is 0.0 points, reflecting a near-ceiling effect and broadly comparable performance overall. This pattern suggests that

TABLE VII

PER-TASK SUCCESS RATES (%) ON ROBOTWIN2.0 (50 TASKS). THE STARRED COLUMNS REPORT THE 50-TASK MIXED-TRAINING SYSTEMS WITH THE SAME LEARNED 64-DIMENSIONAL TASK EMBEDDING: *DP3** OTHERWISE FOLLOWS VANILLA DP3, WHILE *Ours** ADDS THE PROPOSED MOVEMENT-TREND PATHWAY AND BOTTLENECK MODULATION. THE OTHER COLUMNS RETAIN THE OFFICIAL BENCHMARK COMPARISONS. THE BEST RESULT IN EACH ROW IS IN **BOLD** WITH BLUE SHADING.

Task	RDT	π_0	ACT	DP	<i>DP3*</i>	<i>Ours*</i>
<i>Single-Arm Object Handling (7 tasks)</i>						
adjust_bottle	81	90	97	97	95	100
grab_roller	74	96	94	98	82	100
shake_bottle	84	99	63	59	98	100
shake_bottle_horizontally	74	97	74	65	98	100
lift_pot	72	84	88	39	100	94
dump_bin_bigbin	64	83	68	49	66	90
hanging_mug	23	11	7	8	20	30
Category Avg.	67.4	80.0	70.1	59.3	79.9	87.7
<i>Articulated Object Interaction (7 tasks)</i>						
click_alarmclock	61	63	32	61	93	93
click_bell	80	44	58	54	96	99
open_laptop	59	85	56	49	76	83
open_microwave	37	80	86	5	54	64
turn_switch	35	27	5	36	60	66
press_stapler	41	62	31	6	88	78
rotate_QRcode	50	68	1	13	40	77
Category Avg.	51.9	61.3	38.4	32.0	72.4	80.0
<i>Bimanual Handover (2 tasks)</i>						
handover_block	45	45	42	10	76	88
handover_mic	90	98	85	53	95	100
Category Avg.	67.5	71.5	63.5	31.5	85.5	94.0
<i>Pick & Transport (8 tasks)</i>						
move_can_pot	25	58	22	39	85	86
move_pillbottle_pad	8	21	0	1	22	44
move_playingcard_away	43	53	36	47	46	67
move_stapler_pad	2	0	0	1	2	13
pick_diverse_bottles	2	27	7	6	79	78
pick_dual_bottles	42	57	31	24	78	84
put_bottles_dustbin	21	54	27	22	59	63
put_object_cabinet	33	68	15	42	74	68
Category Avg.	22.0	42.3	17.3	22.8	55.6	62.9
<i>Precise Placement (17 tasks)</i>						
place_a2b_left	3	31	1	2	30	48
place_a2b_right	1	27	0	13	36	55
place_bread_basket	10	17	6	14	53	54
place_bread_skillet	5	23	7	11	50	55
place_burger_fries	50	80	49	72	81	66
place_can_basket	19	41	1	18	80	81
place_cans_plasticbox	6	34	16	40	99	96
place_container_plate	78	88	72	41	93	91
place_dual_shoes	4	15	9	8	20	16
place_empty_cup	56	37	61	37	66	91
place_fan	12	20	1	3	18	24
place_mouse_pad	1	7	0	0	1	5
place_object_basket	33	16	15	15	43	59
place_object_scale	1	10	0	1	11	13
place_object_stand	15	36	1	22	60	65
place_phone_stand	15	35	2	13	62	52
place_shoe	35	28	5	23	35	61
Category Avg.	20.2	32.1	14.5	19.6	49.3	54.8
<i>Multi-Object Stacking (4 tasks)</i>						
stack_blocks_two	21	42	25	7	27	35
stack_blocks_three	2	17	0	0	1	3
stack_bowls_two	76	91	82	61	73	88
stack_bowls_three	51	66	48	63	50	67
Category Avg.	37.5	54.0	38.8	32.8	37.8	48.3
<i>Tool Use & Fine-Grained Precision (5 tasks)</i>						
beat_block_hammer	77	43	56	42	70	80
stamp_seal	1	3	2	2	30	30
scan_object	4	18	2	9	31	28
blocks_Ranking_Size	3	19	1	0	3	6
blocks_Ranking_RGB	0	7	0	1	2	5
Category Avg.	17.0	18.0	12.2	10.8	27.2	29.8
Overall Average	34.5	46.4	29.7	28.0	56.1	62.8

movement trend guidance is most beneficial when the base policy has learned a reasonable action prior but lacks explicit

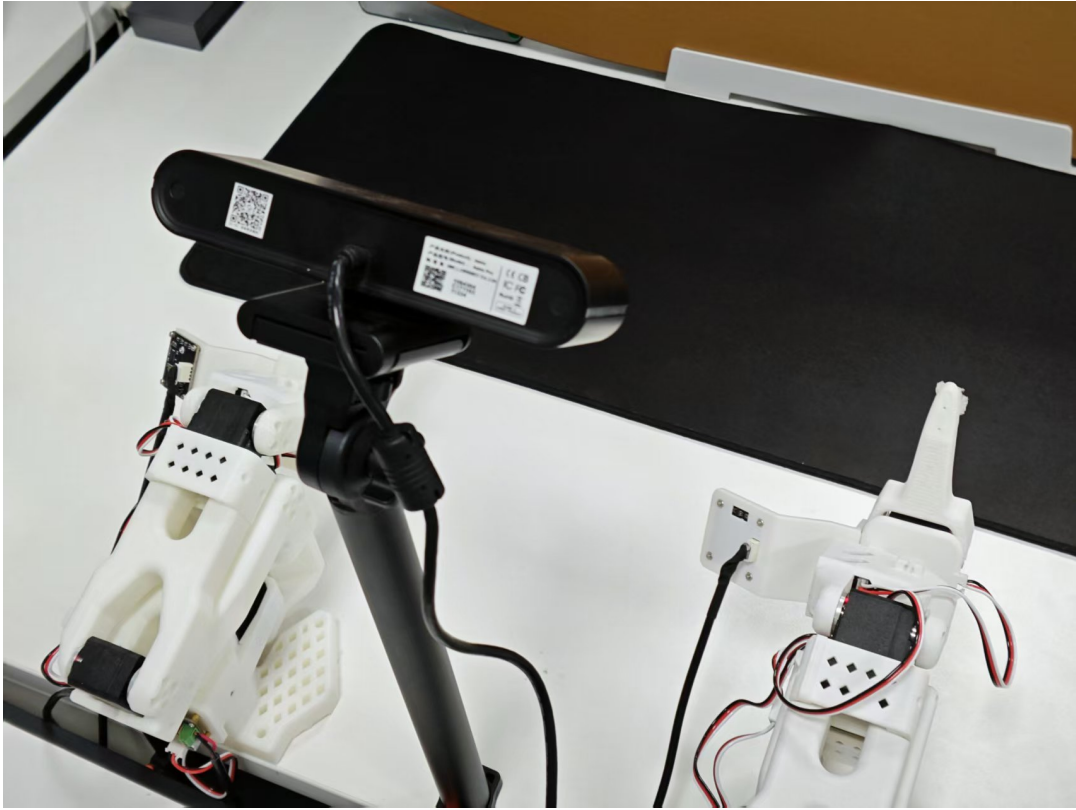


Fig. 5. Real-robot experimental setup with the SO101 dual-arm platform and Intel RealSense RGB-D cameras. Stereo depth is deprojected with camera intrinsics and transformed with calibrated camera-to-robot extrinsics to obtain the policy point cloud.

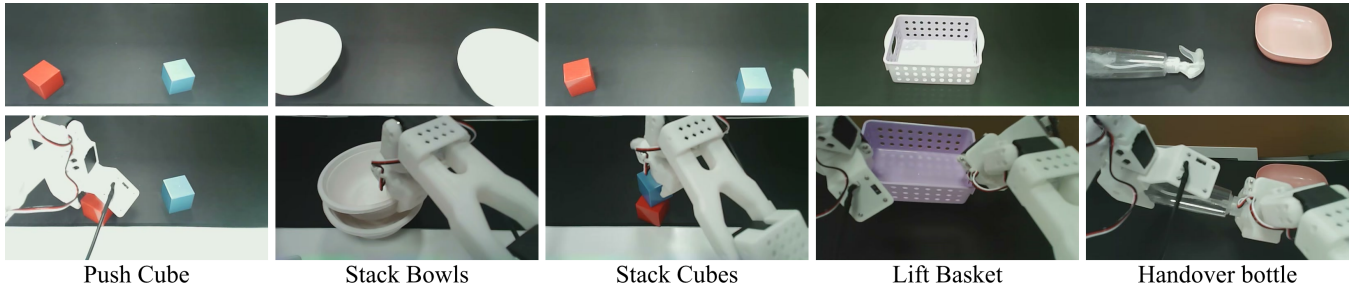


Fig. 6. Real-robot demonstrations on five manipulation tasks.

geometric cues about the future movement trend.

APPENDIX III COMPLETE LIBERO-40 CHECKPOINT RESULTS

Tables VIII and IX report every saved checkpoint in the LIBERO-40 evaluation. Each entry is the suite-level success rate (%) written as mean $\pm$ sample standard deviation across rollout seeds 7, 17, and 27. Each suite contains 10 tasks and 1,500 evaluation episodes per checkpoint; Overall contains all 40 tasks and 6,000 episodes. All values are computed directly from the bounded seed-level rates in the accompanying CSV.

APPENDIX IV DETAILED ABLATION ANALYSIS

The main paper summarizes the ablation results with Fig. 4. Here we provide additional interpretation for each group of variants.

For guidance construction and training, removing gated FiLM reduces the model to a DP3-style policy in which the latent trend is available only through early fusion. The comparison tests whether the latent contributes more than an increase in representation capacity. We also compare parameter-matched variants that replace the default MLP trend predictor with a diffusion predictor or remove input normalization. These variants test whether the result depends on the specific predictor and preprocessing choices. The

end-to-end configuration allows the latent to adapt to the action objective while remaining a soft condition that does not prescribe the executed motion.

For future-target offsets, the observation history is fixed to three frames while the supervised offsets are varied. Short-range offsets can be redundant with the observation window, while long-range offsets provide broader movement information. The selected offsets summarize near-term contact and longer-range movement tendency without creating an intermediate target for the policy to reach.

For injection architecture, applying gated FiLM at every UNet block reduces success from the *Ours** value of 62.8% to 55.8%. This suggests that pervasive gated modulation can make the high-level latent interfere with local action refinement, especially in shallow and upsampling blocks responsible for short-term correction. Replacing gated FiLM with cross-attention performs even worse, reaching 52.9% and falling below the 56.1% *DP3** baseline. Cross-attention creates a stronger coupling between denoising features and guidance tokens, while gated FiLM uses element-wise affine modulation with a learnable gate initialized near identity. The ordering among bottleneck-only gated FiLM, all-block gated FiLM, and cross-attention supports the conservative injection design: latent trend guidance is most useful when it shapes global action structure without dominating local denoising behavior.

APPENDIX V

FULL PER-TASK ABLATION RESULTS ON ROBOTWIN2.0

Tables X, XI, and XII report the complete per-task RoboTwin2.0 ablation results summarized in Figure 4.

TABLE VIII

DP3 LIBERO-40 SUCCESS RATES ACROSS ALL SAVED CHECKPOINTS (MEAN $\pm$ SAMPLE STANDARD DEVIATION ACROSS THREE ROLLOUT SEEDS, %).

Epoch	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-10	Overall
100	0.93 $\pm$ 0.46	3.73 $\pm$ 0.23	1.67 $\pm$ 1.15	1.20 $\pm$ 0.20	1.88 $\pm$ 0.24
200	3.40 $\pm$ 0.20	5.80 $\pm$ 0.20	1.73 $\pm$ 0.31	0.33 $\pm$ 0.31	2.82 $\pm$ 0.10
300	7.67 $\pm$ 1.17	12.47 $\pm$ 2.10	1.87 $\pm$ 0.90	4.67 $\pm$ 1.03	6.67 $\pm$ 0.70
400	6.87 $\pm$ 1.27	7.80 $\pm$ 0.87	6.40 $\pm$ 0.72	9.87 $\pm$ 1.55	7.73 $\pm$ 0.70
500	8.53 $\pm$ 0.81	33.80 $\pm$ 1.06	5.87 $\pm$ 0.50	18.40 $\pm$ 1.64	16.65 $\pm$ 0.35
600	20.73 $\pm$ 1.47	33.13 $\pm$ 2.32	7.33 $\pm$ 0.83	28.93 $\pm$ 2.05	22.53 $\pm$ 0.23
700	18.93 $\pm$ 1.70	46.33 $\pm$ 3.86	8.53 $\pm$ 0.58	31.33 $\pm$ 3.45	26.28 $\pm$ 0.81
800	19.33 $\pm$ 2.12	60.40 $\pm$ 0.20	7.87 $\pm$ 0.61	42.00 $\pm$ 0.20	32.40 $\pm$ 0.43
900	26.93 $\pm$ 1.72	66.13 $\pm$ 0.81	9.93 $\pm$ 1.55	41.47 $\pm$ 0.12	36.12 $\pm$ 0.15
1000	26.40 $\pm$ 1.31	67.53 $\pm$ 1.14	8.73 $\pm$ 0.76	45.67 $\pm$ 1.80	37.08 $\pm$ 1.21

TABLE IX

OURS LIBERO-40 SUCCESS RATES ACROSS ALL SAVED CHECKPOINTS (MEAN $\pm$ SAMPLE STANDARD DEVIATION ACROSS THREE ROLLOUT SEEDS, %).

Epoch	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-10	Overall
100	35.73 $\pm$ 1.50	12.67 $\pm$ 1.14	39.27 $\pm$ 1.97	8.67 $\pm$ 0.23	24.08 $\pm$ 0.74
200	22.33 $\pm$ 1.47	57.00 $\pm$ 2.12	46.07 $\pm$ 1.21	17.87 $\pm$ 1.47	35.82 $\pm$ 0.80
300	65.33 $\pm$ 2.77	88.53 $\pm$ 0.76	75.40 $\pm$ 3.29	48.93 $\pm$ 1.67	69.55 $\pm$ 1.49
400	57.00 $\pm$ 2.11	93.53 $\pm$ 1.21	74.53 $\pm$ 1.29	60.33 $\pm$ 0.99	71.35 $\pm$ 0.74
500	62.53 $\pm$ 3.71	94.20 $\pm$ 0.80	76.07 $\pm$ 2.23	57.20 $\pm$ 1.06	72.50 $\pm$ 1.19
600	60.13 $\pm$ 1.81	94.60 $\pm$ 0.60	76.13 $\pm$ 1.67	59.80 $\pm$ 0.72	72.67 $\pm$ 0.28
700	57.80 $\pm$ 0.87	94.13 $\pm$ 1.01	78.27 $\pm$ 1.86	57.87 $\pm$ 1.03	72.02 $\pm$ 0.29
800	59.07 $\pm$ 0.46	93.20 $\pm$ 1.20	78.40 $\pm$ 0.35	57.93 $\pm$ 1.50	72.15 $\pm$ 0.61
900	58.47 $\pm$ 0.23	94.47 $\pm$ 1.10	76.80 $\pm$ 1.04	59.07 $\pm$ 2.34	72.20 $\pm$ 0.69
1000	58.20 $\pm$ 0.72	93.53 $\pm$ 0.70	77.53 $\pm$ 2.32	58.47 $\pm$ 0.64	71.93 $\pm$ 0.49

TABLE X

PER-TASK SLICES OF THE 50-TASK MIXED-TRAINING ABLATION FOR LATENT GUIDANCE CONSTRUCTION AND TRAINING ON ROBOTWIN2.0 (SUCCESS RATE %). ALL COLUMNS ARE VARIANTS OF A SHARED POLICY TRAINED JOINTLY ON THE FULL SUITE. THE DIFFUSION-PREDICTOR COLUMN REPLACES THE DEFAULT MLP TREND PREDICTOR, MATCHING FIG. 4.

Task	<i>DP3*</i>	<i>Ours*</i>	w/o gate	Diff. pred.	w/o norm.
<i>Single-Arm Object Handling</i>					
adjust_bottle	95	100	100	100	100
grab_roller	82	100	100	99	100
shake_bottle	98	100	100	98	100
shake_bottle_horizontally	98	100	100	99	100
lift_pot	100	94	100	94	97
dump_bin_bigbin	66	90	89	86	89
hanging_mug	20	30	21	33	19
<i>Articulated Object Interaction</i>					
click_alarmclock	93	93	91	88	93
click_bell	96	99	100	99	99
open_laptop	76	83	77	75	83
open_microwave	54	64	68	94	67
turn_switch	60	66	47	56	55
press_stapler	88	78	65	75	72
rotate_QRcode	40	77	52	62	61
<i>Bimanual Handover</i>					
handover_block	76	88	93	90	94
handover_mic	95	100	100	97	100
<i>Pick & Transport</i>					
move_can_pot	85	86	79	79	85
move_pillbottle_pad	22	44	43	35	47
move_playingcard_away	46	67	68	62	73
move_stapler_pad	2	13	9	14	8
pick_diverse_bottles	79	78	62	59	65
pick_dual_bottles	78	84	65	53	54
put_bottles_dustbin	59	63	67	53	68
put_object_cabinet	74	68	56	60	70
<i>Precise Placement</i>					
place_a2b_left	30	48	26	31	38
place_a2b_right	36	55	21	41	40
place_bread_basket	53	54	28	27	29
place_bread_skillet	50	55	29	29	30
place_burger_fries	81	66	65	63	77
place_can_basket	80	81	74	68	78
place_cans_plasticbox	99	96	69	70	62
place_container_plate	93	91	92	90	86
place_dual_shoes	20	16	16	17	12
place_empty_cup	66	91	90	78	83
place_fan	18	24	42	33	39
place_mouse_pad	1	5	6	4	6
place_object_basket	43	59	62	56	65
place_object_scale	11	13	16	14	13
place_object_stand	60	65	50	51	53
place_phone_stand	62	52	41	57	52
place_shoe	35	61	51	54	46
<i>Multi-Object Stacking</i>					
stack_blocks_two	27	35	25	27	33
stack_blocks_three	1	3	3	4	2
stack_bowls_two	73	88	92	93	91
stack_bowls_three	50	67	79	72	75
<i>Tool Use & Fine-Grained Precision</i>					
beat_block_hammer	70	80	74	68	76
stamp_seal	30	30	25	29	21
scan_object	31	28	24	18	18
blocks_Ranking_Size	3	6	3	3	3
blocks_Ranking_RGB	2	5	1	7	2
Average	56.1	62.8	57.1	57.3	58.6

TABLE XI

PER-TASK SLICES OF THE 50-TASK MIXED-TRAINING ABLATION FOR FUTURE-TARGET OFFSET CONFIGURATIONS ON ROBOTWIN2.0 (SUCCESS RATE %).

THE OBSERVATION HISTORY IS FIXED TO THREE FRAMES; COLUMN NAMES MATCH FIG. 4. ALL COLUMNS ARE VARIANTS OF A SHARED POLICY TRAINED JOINTLY ON THE FULL SUITE.

Task	DP3*	Ours*	7 offsets	short range	long range	2 offsets	single near	single far
<i>Single-Arm Object Handling</i>								
adjust_bottle	95	100	100	100	85	99	100	100
grab_roller	82	100	98	99	100	100	100	100
shake_bottle	98	100	99	96	96	96	98	99
shake_bottle_horizontally	98	100	97	98	100	99	98	98
lift_pot	100	94	90	91	96	91	91	92
dump_bin_bigbin	66	90	84	89	91	86	82	81
hanging_mug	20	30	21	19	19	26	30	26
<i>Articulated Object Interaction</i>								
click_alarmclock	93	93	80	78	94	88	84	80
click_bell	96	99	100	99	97	99	100	99
open_laptop	76	83	83	83	82	87	80	81
open_microwave	54	64	58	51	59	63	60	85
turn_switch	60	66	55	53	52	59	51	54
press_stapler	88	78	77	74	75	73	71	72
rotate_QRcode	40	77	50	74	52	57	55	74
<i>Bimanual Handover</i>								
handover_block	76	88	92	85	89	83	91	87
handover_mic	95	100	91	97	66	97	99	100
<i>Pick & Transport</i>								
move_can_pot	85	86	88	86	86	81	81	83
move_pillbottle_pad	22	44	33	37	40	34	34	24
move_playingcard_away	46	67	71	73	83	70	74	71
move_stapler_pad	2	13	17	10	4	12	4	8
pick_diverse_bottles	79	78	54	52	68	62	54	55
pick_dual_bottles	78	84	56	54	73	67	70	57
put_bottles_dustbin	59	63	62	61	72	58	54	49
put_object_cabinet	74	68	68	65	68	59	68	57
<i>Precise Placement</i>								
place_a2b_left	30	48	30	27	39	31	30	26
place_a2b_right	36	55	40	38	41	34	40	38
place_bread_basket	53	54	32	20	28	16	18	22
place_bread_skillet	50	55	35	30	31	31	33	27
place_burger_fries	81	66	70	72	70	68	74	73
place_can_basket	80	81	76	73	65	73	74	76
place_cans_plasticbox	99	96	84	56	69	69	56	61
place_container_plate	93	91	80	86	91	85	83	87
place_dual_shoes	20	16	14	10	13	17	17	11
place_empty_cup	66	91	80	81	86	81	76	69
place_fan	18	24	37	39	41	45	42	33
place_mouse_pad	1	5	2	3	5	1	4	3
place_object_basket	43	59	60	54	64	50	48	52
place_object_scale	11	13	8	13	11	7	9	6
place_object_stand	60	65	61	50	55	49	51	52
place_phone_stand	62	52	56	42	49	43	39	48
place_shoe	35	61	48	46	59	51	45	51
<i>Multi-Object Stacking</i>								
stack_blocks_two	27	35	24	27	24	24	31	29
stack_blocks_three	1	3	1	2	2	0	1	2
stack_bowls_two	73	88	90	92	90	80	78	88
stack_bowls_three	50	67	60	68	78	68	62	69
<i>Tool Use & Fine-Grained Precision</i>								
beat_block_hammer	70	80	66	66	63	76	61	52
stamp_seal	30	30	29	29	25	23	22	27
scan_object	31	28	19	16	21	19	19	23
blocks_Ranking_Size	3	6	3	3	3	1	1	1
blocks_Ranking_RGB	2	5	2	2	2	2	3	1
Average	56.1	62.8	56.6	55.4	57.4	55.8	54.9	55.2

TABLE XII

PER-TASK SLICES OF THE 50-TASK MIXED-TRAINING ABLATION FOR MOVEMENT-TREND GUIDANCE INJECTION ON ROBOTWIN2.0 (SUCCESS RATE %).
ALL COLUMNS ARE VARIANTS OF A SHARED POLICY TRAINED JOINTLY ON THE FULL SUITE.

Task	<i>DP3*</i>	<i>Ours*</i>	All-block	Cross-attn
<i>Single-Arm Object Handling</i>				
adjust_bottle	95	100	100	0
grab_roller	82	100	99	99
shake_bottle	98	100	98	95
shake_bottle_horizontally	98	100	100	98
lift_pot	100	94	100	93
dump_bin_bigbin	66	90	80	80
hanging_mug	20	30	20	23
<i>Articulated Object Interaction</i>				
click_alarmclock	93	93	79	90
click_bell	96	99	99	99
open_laptop	76	83	82	77
open_microwave	54	64	75	71
turn_switch	60	66	53	54
press_stapler	88	78	76	70
rotate_QRcode	40	77	75	76
<i>Bimanual Handover</i>				
handover_block	76	88	78	98
handover_mic	95	100	89	96
<i>Pick & Transport</i>				
move_can_pot	85	86	81	85
move_pillbottle_pad	22	44	44	22
move_playingcard_away	46	67	78	67
move_stapler_pad	2	13	8	7
pick_diverse_bottles	79	78	48	55
pick_dual_bottles	78	84	72	41
put_bottles_dustbin	59	63	55	60
put_object_cabinet	74	68	63	65
<i>Precise Placement</i>				
place_a2b_left	30	48	21	35
place_a2b_right	36	55	41	36
place_bread_basket	53	54	24	23
place_bread_skillet	50	55	31	25
place_burger_fries	81	66	65	60
place_can_basket	80	81	70	69
place_cans_plasticbox	99	96	38	52
place_container_plate	93	91	82	86
place_dual_shoes	20	16	10	14
place_empty_cup	66	91	89	84
place_fan	18	24	38	42
place_mouse_pad	1	5	5	2
place_object_basket	43	59	61	52
place_object_scale	11	13	10	8
place_object_stand	60	65	52	52
place_phone_stand	62	52	46	48
place_shoe	35	61	63	44
<i>Multi-Object Stacking</i>				
stack_blocks_two	27	35	22	28
stack_blocks_three	1	3	1	2
stack_bowls_two	73	88	86	87
stack_bowls_three	50	67	61	73
<i>Tool Use & Fine-Grained Precision</i>				
beat_block_hammer	70	80	63	57
stamp_seal	30	30	21	28
scan_object	31	28	29	9
blocks_Ranking_Size	3	6	4	3
blocks_Ranking_RGB	2	5	5	7
Average	56.1	62.8	55.8	52.9